

Local-CI-Production workflow with DrevOps

DrupalSouth Hobart 2019



The problem



Starting a new project

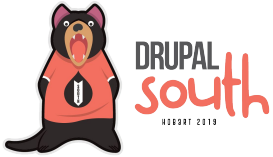
- Scaffold Drupal
- Configure tools (code linting, testing frameworks etc)
- Configure CI
- Configure deployment
- Write documentation



Onboarding to existing project

- Read documentation (if any)
- Get database
- Configure local environment
- Configure tools (code linting, testing frameworks etc)
- Configure CI
- Configure deployment





Potential solutions



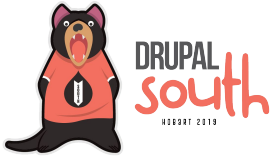
Potential solutions

- Do it from scratch every time
- Copy configs from the previous project and adapt
- Use scaffolding tools + custom code and adapt
- Use project template


OVER
TIME



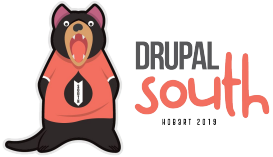






https://github.com/drevops/drevops





What is DrevOps?

Local-CI-Production workflow with DrevOps



Local-CI-Production workflow with DrevOps



What is DrevOps

- Drupal project scaffolding template
- Docker stack configuration
- CI configuration and tools
- Production hosting integration configuration
- Documentation repository
- “Glue code” to keep it all together



What DrevOps is NOT

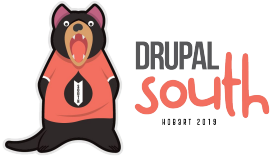
- A replacement for Lando, DDEV, Tokaido etc.
- A (yet another) custom docker images repository
- A hosting provider
- A CI system
- Paid SaaS or PaaS service



The approach

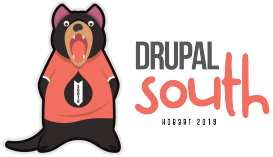
Identical environments





Where DrevOps is already used

- Around 30+ custom websites
- In similar form - GovCMS 2.0 SaaS and PaaS
- In similar form - Single Digital Presence



Features

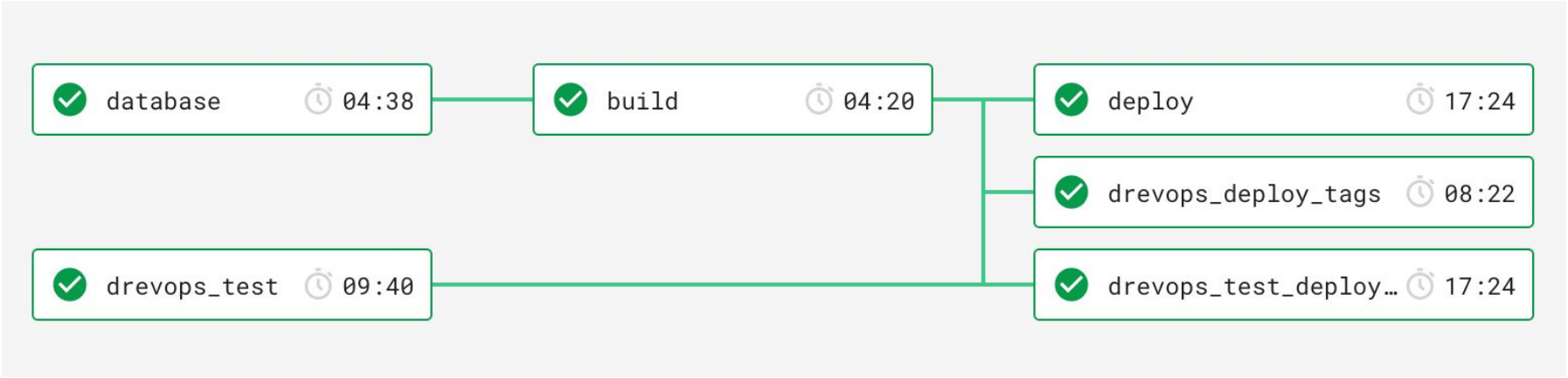


Features

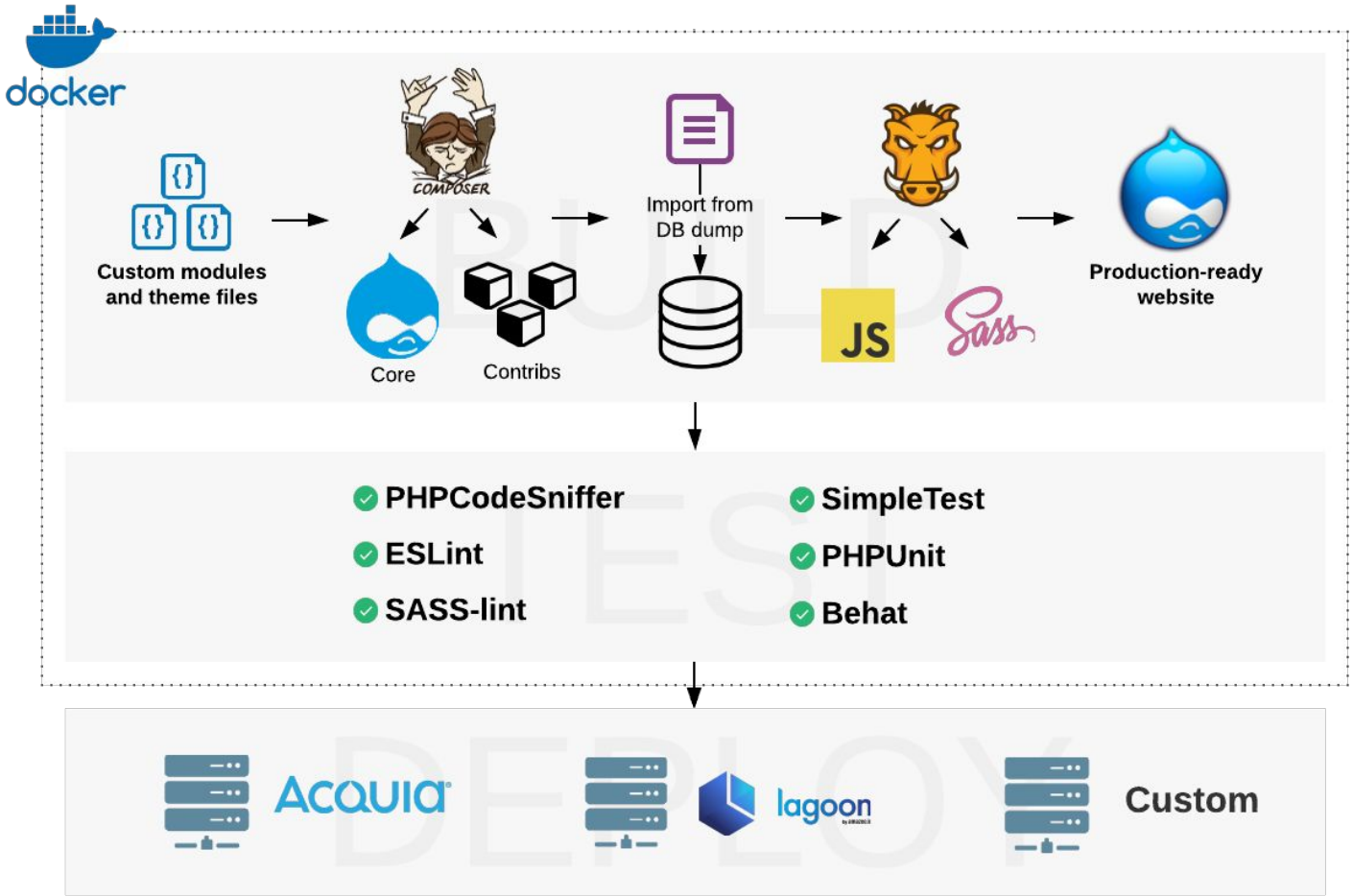
- “Pure” docker
- Flexible configuration using variables
- Identical commands in all environments
- Self-tested
- Updatable and versionable

Stability

DrevOps runs end-to-end test and *build->test->deployment* on every commit



Local-CI-Production workflow with DrevOps


docker
Custom modules and theme files
COMPOSER
Core
Contribs
Import from DB dump
JS
Sass
Production-ready website
BUILD
PHPCodeSniffer
ESLint
SASS-lint
SimpleTest
PHPUnit
Behat
TEST
Acquia
lagoon
Custom
DEPLOY

Building in Docker

Building in Docker

Build:

1. Pull images from Docker Hub
2. Create images for services with custom Dockerfiles
3. Install code dependencies
4. Add application source code
5. Build assets

Runtime:

1. Start container from locally built images
2. Volumes are shared between containers
3. **Volumes are shared between containers and the host (Local only)**

Implementing workflow actions


SCRIPTS
LIGHTYEAR
SCRIPTS EVERYWHERE



Workflow actions

- Bash scripts
- Behaviour is controlled by variables
- Run inside containers
- Used in all environments



Why Bash scripts?

- Portable (POSIX compliant)
- Simple to maintain
- Direct use or through wrapper like Ahoy or Make
- Cheat sheet available in DrevOps



Command wrapper



Ahoy

Workflow scripts are called from Ahoy (ahoycli.com)

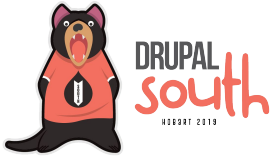
- YAML for commands
- Configurable entrypoint
- Runs commands from any subdirectory of the project

```yaml
start:
  usage: Start existing Docker containers.
  cmd: docker-compose start "$@"

stop:
  usage: Stop running Docker containers.
  cmd: docker-compose stop "$@"

restart:
  usage: Restart stopped and running Docker containers.
  cmd: docker-compose restart "$@"

logs:
  usage: Show Docker logs for services.
  cmd: docker-compose logs "$@"
```



Ahoy vs Make

```yaml
start:
  usage: Start existing Docker containers.
  cmd: docker-compose start "$@"

stop:
  usage: Stop running Docker containers.
  cmd: docker-compose stop "$@"

restart:
  usage: Restart stopped and running Docker containers.
  cmd: docker-compose restart "$@"

logs:
  usage: Show Docker logs for services.
  cmd: docker-compose logs "$@"
```

```
.DEFAULT_GOAL := help
.PHONY: start stop restart logs
.EXPORT_ALL_VARIABLES: ;

start:
    $(call title,Starting Docker containers)
    $(call exec,docker-compose up -d $(filter-out $@,$(MAKECMDGOALS)))

stop:
    $(call title,Stopping Docker containers)
    $(call exec,docker-compose stop $(filter-out $@,$(MAKECMDGOALS)))

restart:
    $(call title,Restarting Docker containers)
    $(call exec,docker-compose restart)

logs:
    $(call title,Displaying Docker logs)
    $(call exec,docker-compose logs)
```



Commands



Build

```
ahoy build
```

- Builds images and starts containers
- Installs development dependencies
- Installs site



Start/Stop stack

```
ahoy up
```

```
ahoy down
```

```
ahoy start
```

```
ahoy stop
```



Site install

```
ahoy install-site
```

- Fresh install from profile
- Install from canonical database
- Runs post-install commands (cim+updb+cr)



Database download

```
ahoy download-db
```

- FTP
- CURL
- Acquia Cloud



Deployment

```
ahoy deploy
```

- Webhook
- Code
- Docker Image



Lint

`ahoy lint`

- Drupal PHP coding standards
- Drupal JavaScript coding standards
- SCSS coding standards



Test

```
ahoy test-unit
```

```
ahoy test-kernel
```

```
ahoy test-functional
```

```
ahoy test-bdd
```



Utility

```
ahoy cli
```

```
ahoy drush
```

```
ahoy login
```



Cleanup

```
ahoy clean
```

```
ahoy reset
```

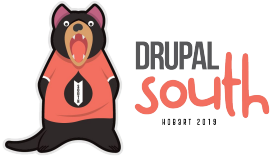


Front-end

```
ahoy fe
```

```
ahoy fed
```

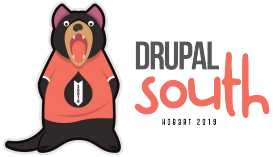
```
ahoy few
```



Xdebug

`ahoy debug`

```
$ ahoy debug
Recreating your_site_cli_1 ... done
Recreating your_site_php_1 ... done
Enabled debug confguration. Use 'ahoy up' to disable.
```

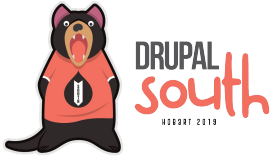


Doctor

`ahoy doctor`

- Diagnoses local stack issues
- Runs before and after the project is built

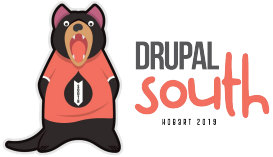
```
+ Checking project requirements
  ✓ All required tools are present
  ✓ Port 80 is available
  ✓ Pygmy is running
  ✓ CLI container is running
  ✓ SSH key is available within CLI container
  ✓ Web server is running and accessible at http://your-site.docker.amazee.io
  ✓ Successfully bootstrapped website at http://your-site.docker.amazee.io
+ All required checks have passed
```



Self-update

```
ahoy update
```

- Fetches the latest stable version of DrevOps
- Changes reviewed and selectively committed



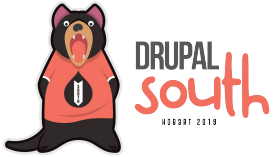
Integrations

- Amazeeio Lagoon
- Acquia Cloud
- dependencies.io



Help with monthly maintenance

- Drupal core
- Contrib modules
- Development stack
- Production stack
- CI configs
- Workflow scripts



Documentation

Local-CI-Production workflow with DrevOps



Documentation

- Readme.md template with essential information
- FAQs template
- Deployment template
- Onboarding checklist

Demo


YOU GET A DEMO
AND YOU GET A DEMO
imgflip.com

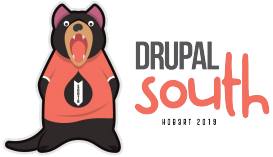


Local-CI-Production workflow with DrevOps



Challenges of maintaining DrevOps

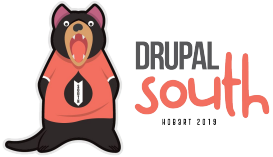
- Keep testing code and code-under-test in the same codebase
- Keep Drupal 7 branch up-to-date with Drupal 8 branch
- Keep code robust, but clear to understand to newcomers
- Provide an easy and clear way to keep the stack up-to-date with the latest version of DrevOps
- Decide how much Drupal functionality should come out of the box
- Testing of workflows is not simple: using Bats, but still adding custom assertions



What's next?

- `composer create project`
- Documentation at readthedocs.org
- Accessibility testing with a11y
- Gulp for front-end
- Nightly dependencies updates in CI
- More integrations: Pantheon, Platform.sh
- More CI providers: Travis CI, GitLab, GitHub actions

How can it help you



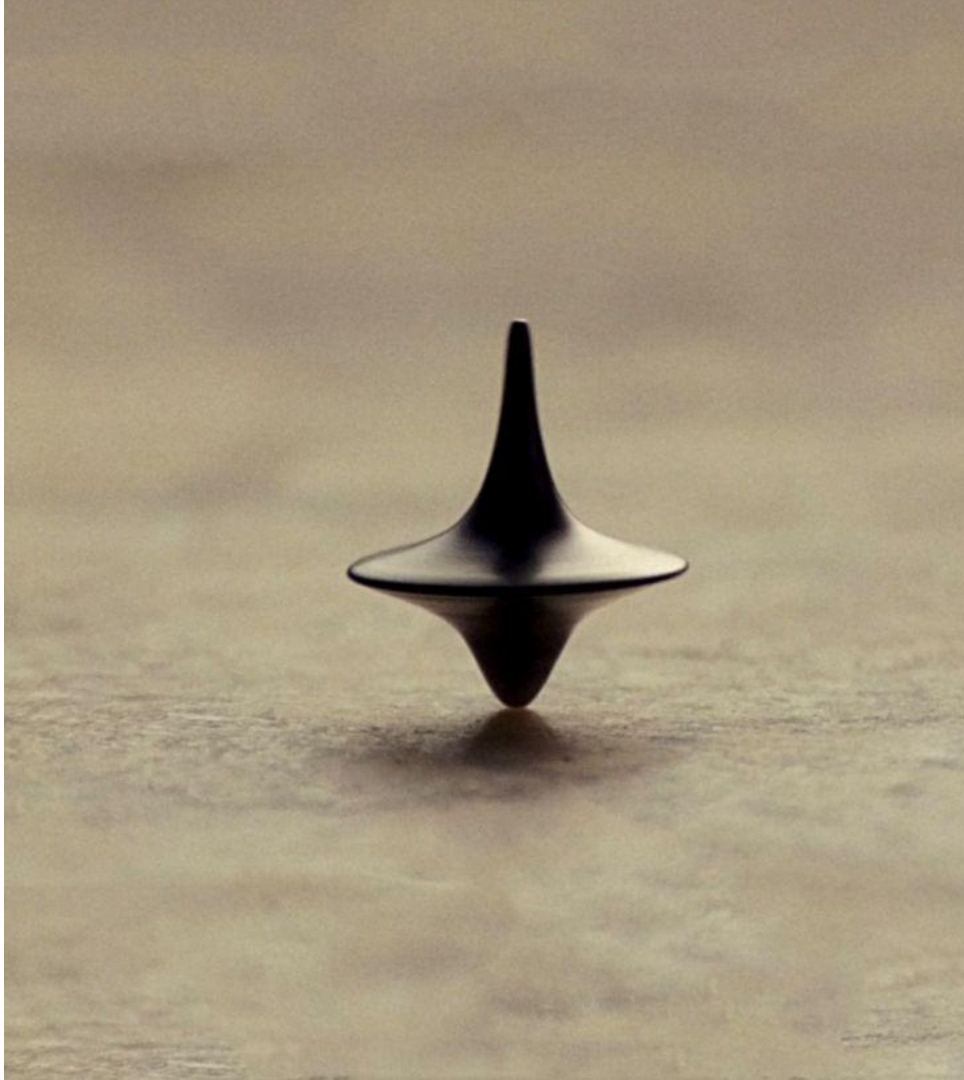
If you are...

- A digital agency looking for **standard operating environment**
- A developer interested in **best practices**
- A developer without required **time** or **knowledge**



A template for your template

Fork and update a handful of variables to maintain your own DrevOps template





Thank you!

@AlexSkrypnyk

alex@integratedexperts.com

drevops.com